本章路線

這章要回答的核心問題是：檔案系統如何以有限 block、metadata 與故障恢復協定，維持可存取且可恢復的持久物件？

前置知識。 file interface、邏輯/實體 block、journaling 概念。

- 追蹤 inode、directory、open table 與 page cache
- 比較 contiguous/linked/indexed allocation
- 分析故障時 metadata 與資料的一致性

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從 logical file 到可恢復的 disk structures

固定教學 volume 的 block size 為 4 KiB：

block	persistent structure	關鍵內容
0	superblock	block size、volume block count、inode/free-space 位置
1	inode bitmap	inode 是否配置
2	data-block bitmap	data block 是否 free
3	inode table	file type、size、mode、direct/indirect pointers
4	root directory	`home→inode 3`
5	`/home` directory	`u→inode 5`
6	`/home/u` directory	`notes.txt→inode 7`
8–10	file data	`notes.txt` 的三個 4 KiB blocks

basic file system 把 logical block 轉成 device block；file-organization module 管理 free-space 與 logical-to-physical mapping；logical file system 管理 directory、FCB/inode 與 API；I/O control 與 driver 完成 block transfer。cache 可放在 upper layer，但 commit 的 durability point 必須由系統契約明確決定。

從 logical file 到可恢復的 disk structures （續）

一次 append 必須維持：

$$\text{name} \to \text{valid inode} \to \text{self-consistent block set}, \qquad \text{allocated} \cap \text{free} = \varnothing.$$

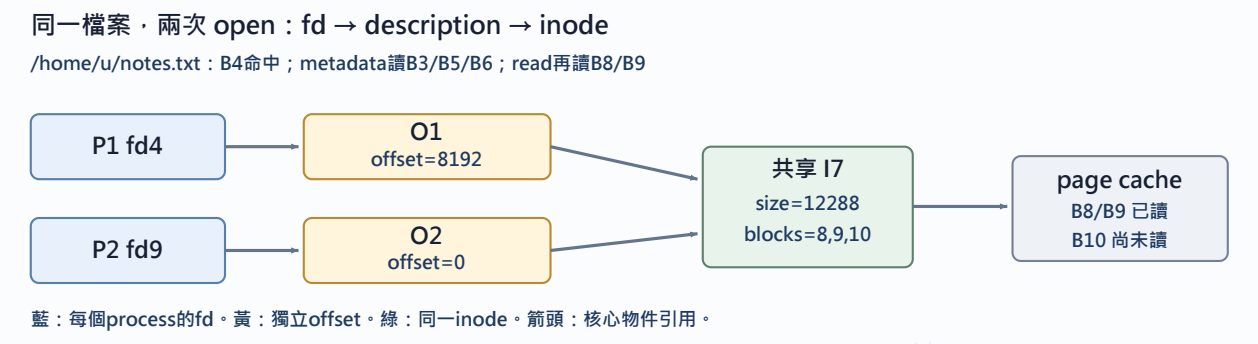
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Path walk、open 與 read 的完整狀態

固定 namespace `/home/u/notes.txt`，初始只快取 root directory block4。cold metadata path：

step	查詢	讀取或命中	得到
1	`/` 中 `home`	root directory block4 hit	inode 3
2	inode 3	inode table block3 cold read	directory block5
3	`/home` 中 `u`	directory block5 cold read	inode 5
4	`/home/u` 中 `notes.txt`	directory block6 cold read；inode table 已快取	inode 7
5	open inode7，權限通過	不新增 disk read	O1→I7、P1 fd4→O1、offset0
6	P1 read 8192 B	data blocks8、9	data加載；P1 offset=8192
7	P2 另一次 open 同 file，權限通過	inode/cache hit	新 O2→I7、P2 fd9→O2、offset0

Path walk、open 與 read 的完整狀態 （續）



這一條 cold path 有3個 metadata block reads（inode table、directory5、directory6）與2個 data block reads，共5次 block read。全部 cached 時是0次 disk read。directory cache、inode cache、system-wide open-file table與 page cache 是不同層，任何一層 hit 都不能直接推出資料持久。

本章區分三層。**fd table** 是每個 process 的整數fd→description引用，可含 close-on-exec 等 descriptor flags。**open-file description** 是 open 建立的核心物件，保存 current offset、access/status flags 及 inode 引用。**inode/FCB** 保存檔案身分、size、權限、block mapping，與共享 page cache 關聯。兩次獨立 open 產生 O1/O2，不能因 inode 相同就把兩者的 offset 合併。

Path walk、open 與 read 的完整狀態 （續）

OSC 9e 的教學圖把部分欄位畫在 per-process table，另有 system-wide FCB 表；若採該抽象，必須另說明共享 offset 所屬的物件。本例採用 Linux/POSIX open/dup 的三層表示，避免把「同一 inode」誤等同「同一 open description」。現在有2個 descriptions 指向 I7；並非同一個 description 的 reference count 等於2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

獨立 open 與 dup 的 offset 不變量

沿用 P1 fd4→O1、P2 fd9→O2，I7 size=12288。每次 read 完整成功、不受 signal 中斷，沒有其他 writer。

事件	fd 關係	O1 offset	O2 offset	回傳範圍
兩次 open 後	P1:4→O1；P2:9→O2	0	0	—
P1 read(fd4,8192)	不變	8192	0	bytes [0,8192)
P1 fd6=dup(fd4)	P1:6 也指 O1	8192	0	不讀資料
P1 read(fd6,4096)	fd4/fd6 共用 O1	12288	0	bytes [8192,12288)
P2 lseek(fd9,4096,SEEK_SET)	只改 O2	12288	4096	不改檔案內容
P1 read(fd4,1)	O1 已在 EOF	12288	4096	回傳0，不再前進

成功讀 r bytes 的一般規則是 $o' = o + r$，其中 $0 \le r \le \min(n, \max(0, size - o))$。`dup`/繼承同一 description 的 `fork` 分享此 o；獨立 open 不分享。關閉 fd6 只減少 O1 引用，fd4 仍可用；最後 description 關閉也不等於移除目錄名稱，unlink 與最後 open reference 是不同生命週期。

Mount 與 VFS 的具體分派

固定 mount table 將根檔案系統 A 的 `/mnt` 掛上檔案系統 B 的 root inode2。解析 `/mnt/log/x` 時，先在 A.root 找到 mountpoint；再切到 `(B,inode2)`，由 B 的 lookup 找 log、x。inode number2 在 A/B 可以重複，故識別至少要包含 filesystem/mount context 與 inode，不能單靠數字。

VFS 先查 dentry/inode 與 mount 關係，對 regular file 的 read 再呼叫該 filesystem 的 operations；local filesystem 解析 block mapping 後送 block I/O，NFS 則形成 RPC。共用介面不是所有下層都以同一個磁碟 index block 實作。權限、symlink、`..` 與 mount traversal 必須在各元件解析時處理，本例的5次讀取模型明定沒有 symlink、mount crossing 或額外權限資料 I/O。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Directory：linear list、hash 與可驗證查找成本

假設 n 個現存名稱被等機率查詢，且每次名稱比較成本相同，linear list 的成功搜尋平均為 $(n+1)/2$ 次比較；若第 i 個名稱查詢機率為 p_i，平均比較次數為 $\Sigma_i\, p_i i$。hash table 把 name 映射至 bucket，再比較 bucket 內 entries。固定 hash：

$$h(name) = \left(\sum_i code(char_i) \right) \bmod 8.$$

七個 names 的分桶：

bucket	entries	chain length
0	—	0
1	—	0
2	delta, epsilon, eta	3
3	gamma	1
4	beta, zeta	2
5	—	0
6	alpha	1
7	—	0

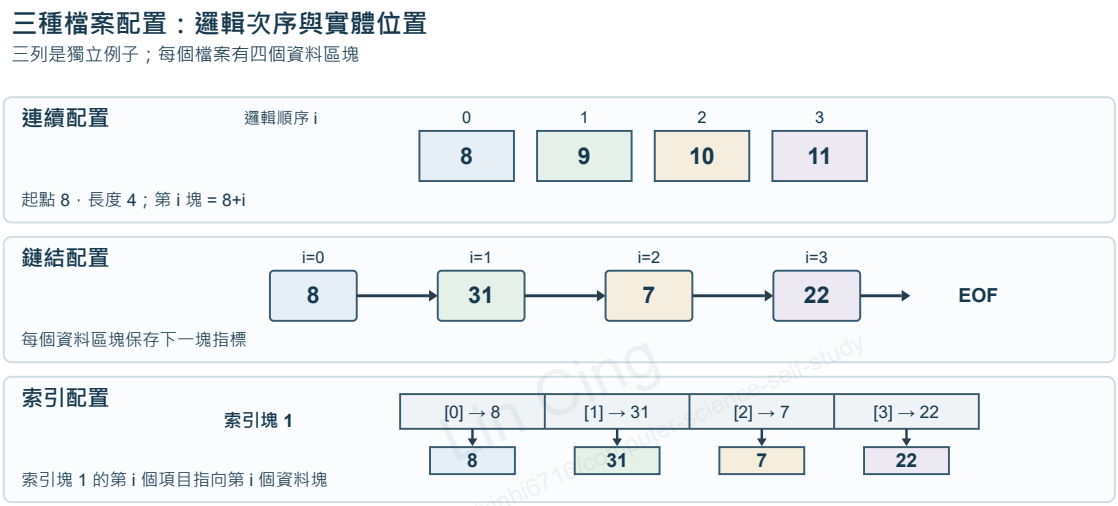
Directory：linear list、hash 與可驗證查找成本 （續）

load factor $\alpha = 7/8 = 0.875$。假設七個現存 keys 等機率被查詢，固定這組 buckets 的成功比較平均是 `(1+1+1+1+2+2+3)/7=11/7=1.571`。這是對查詢目標取平均，沒有對 hash function 取期望。

若另假設每對相異 keys 獨立、等機率碰撞到 m 個 buckets，鏈結位置依固定插入順序排列，則第 i 個插入者的期望成功比較為 $1 + (i-1)/m$。對 $i = 1 \ldots n$ 平均，得 $1 + (n-1)/(2m)$；本例 n=7,m=8 為1.375。常寫的 $1 + \alpha/2$ 是忽略有限 n 校正的近似。觀測1.571與期望1.375的分母/隨機性不同，不能說前者算錯。若所有 keys 碰撞，最壞仍為 $O(n)$。

chaining 的8 buckets 可存超過8個 names，不會因第9個 name 就必然裝不下；可依 load factor 與延遲政策選擇 resize。open addressing 則每 slot 至多一 entry，容量限制不同。rehash 的中間版本若讀者可見，可能漏掉舊 bucket 或出現同名兩筆；正常協定需用鎖、版本或增量搬移規則保證每次查找有一致的可見結果。

三種資料區塊配置



Allocation methods：直接、鏈結與索引的成本

固定 block=4 KiB、pointer=4 bytes，因此每個 pointer block/indirect block有1024 entries。以 logical block2（0-based）為例：

method	mapping state	取得 logical block2 的 disk reads	主要代價
contiguous	start=11, length=3	1 data read	external fragmentation、成長需搬移
linked	11→12→13	3 block reads	random access要追鏈
indexed，index cached	inode/index entry→data13	1 data read	index space
indexed，index cold	read index block，再讀 data13	2 reads	大型 index memory/TLB

contiguous block i 的映射為 `start+i`；linked 必須從 chain head 走；indexed 可用第 i pointer，代價取決於 direct、single、double、triple indirect 層級。

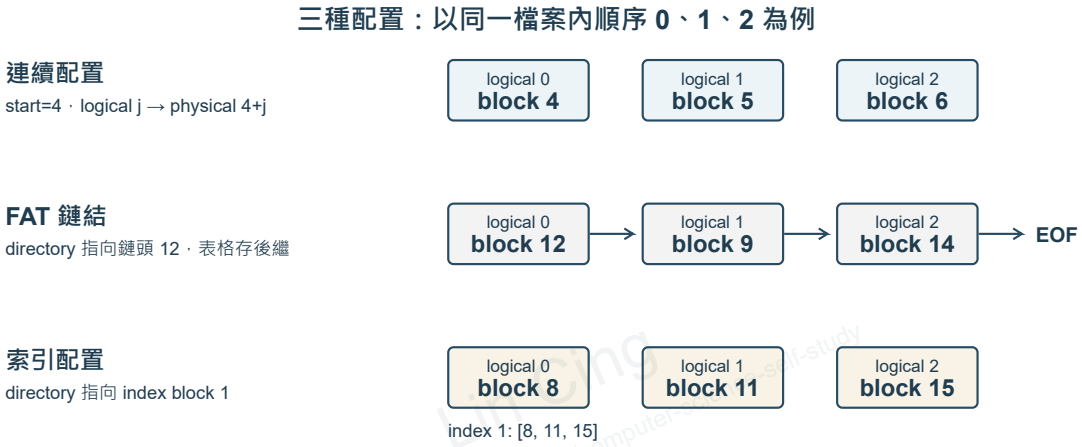
Allocation methods：直接、鏈結與索引的成本 （續）

12 direct pointers、1024-entry indirect blocks的四個範圍：

區間	logical block覆蓋	data bytes
direct	0–11	48 KiB
single indirect	12–1035	4 MiB
double indirect	1036–1,049,611	4 GiB
triple indirect	再下一段	4 TiB

若 offset=5 MiB，已超出12 direct+single indirect範圍，需要 double indirect；sparse hole 不必配置 data block，read 回零，但 logical size 可涵蓋該 offset。把「inode pointer 為0」解成 file 損壞會誤判合法 sparse file。

三種配置：從檔案內位置找到實體區塊



這是三個獨立的配置狀態；圖中的數字為 physical block 編號。

三個例子互不接續，各有 3 個 data blocks，logical block 從 0 開始。連續配置的 `start=4, length=3` 將 logical block 2 算成 physical block `4+2=6`。FAT 的 directory 記錄鏈頭 12，查 FAT 得 `12→9→14→EOF`；要找 logical block 2，沿鏈取得 12 的後繼 9，再取得 9 的後繼 14。索引配置的 directory 指向 index block 1，其三個 entry 依序記錄 8、11、15，所以 logical block 2 指向 physical block 15。

兩層索引：地址轉換與 I/O 分開計算

固定每個 data block 與 index block 均為 4 KiB，每個 block pointer 為 4 bytes，因此每張索引有 `N=4096/4=1024` 筆。兩層索引最多選到 $N^2=1{,}048{,}576$ 個 data blocks；logical block j（從 0 起算）使用 `outer=floor(j/N)`、`inner=j mod N`。

例如 `j=1280`：outer=1、inner=256。先從第一層 entry 1 找第二層 index block，再從其 entry 256 找資料區塊。若 inode 已在記憶體、兩層 index blocks 和目標 data block 均不在相關快取、每個都需一次讀取，且不計其他 metadata 與 device cache，這次讀取模型是 **2 次 index-block reads + 1 次 data-block read = 3 次**。若第一層已快取，則少一次 index read；若 data 亦已快取，所需 I/O 會再減少。這是指定模型的讀取次數，而非任何實作一律固定三次。

inode 索引樹的容量上限

假設 inode 有 12 個 direct pointers，另有各 1 個 single、double、triple indirect pointer；block 大小 4 KiB，每個 pointer 4 bytes，所以一張 index block 可放 `N=1024` 個 pointers。每個 direct pointer 指一個 data block；其餘三種依序可指向 N、N^2、N^3 個 data blocks。

路徑	可定位 data blocks
12 個 direct	12
single indirect	1,024
double indirect	1,048,576
triple indirect	1,073,741,824
合計	**1,074,791,436**

容量為 `1,074,791,436 × 4096 = 4,402,345,721,856 bytes`，約 **4.00391 TiB**。這是本索引樹可定址資料區塊的結構上限；尚未扣除索引 metadata 的實體儲存需求，亦不代表檔案系統的 file-size 欄位、block address 格式或實體容量一定允許如此大的檔案。若檔案有 hole，在檔案大小內讀取 hole 得零值資料；在 EOF 之後讀取則回傳 0 bytes，不能混為一談。

由 byte offset 解出 indirect 索引

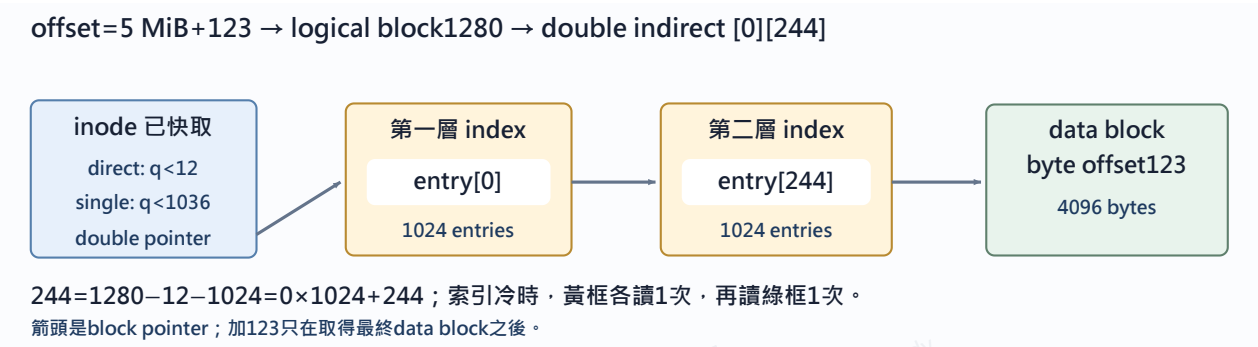
令 $q = \lfloor offset/4096 \rfloor, d = offset \bmod 4096$。前12個 logical blocks 是 direct；其後1024個是 single；再其後 1024^2 個是 double；最後 1024^3 個是 triple。邊界使用左閉右開區間，避免最大索引多算1。

固定 `offset=5 MiB+123=5,243,003` 且該位元組位於檔案 size 以內，有 $q = 1280, d = 123$。先扣 direct 的12，再扣 single 的1024，得 double 範圍內編號244。用1024進位表示：

$$244 = 0 \cdot 1024 + 244.$$

inode.double→第一層 entry0→第二層 entry244→data frame，再加 byte123。若 inode 已在記憶體、兩層 index 與 data 都冷，讀取恰為2個 index blocks+1個 data block；若第一層 index 已快取，則少一次 metadata I/O。

由 byte offset 解出 indirect 索引 （續）



總 data addressability 為 $4096(12 + 1024 + 1024^2 + 1024^3)$ bytes。這沒有把 index blocks 自己算成使用者資料；實際最大 file size 還受 size 欄位、block number 範圍及 filesystem 規格限制。在檔案 size 以內，sparse hole 的 pointer=0 可回傳零而不配置 data block；讀回的零不證明磁碟上存在全零頁。若讀取起點已在 EOF，則一般 read 回傳 0 bytes，不會無限回傳零。

配置不變量須容許合法的 **sparse holes** 與 **reflink/copy-on-write sharing**。hole代表沒有data block，不能判成失去指標；reflink讓不同inode暫時引用同一受COW保護的block，另以引用或版本資料追蹤，在任一方修改時才分離。沒有共享協定卻把同一block當成兩個互相獨立的可寫配置，才是重複配置錯誤；一份inode停止引用也不能在其他snapshot/reflink仍使用時就把block加入free set。

FAT 與嵌入式 linked allocation 的差別

固定 chain `11→12→13→EOF`。若 next pointer 就放在每個4 KiB data block，4-byte pointer 留下 4092 bytes payload；要求第3個 chunk 要先讀11、12取得鏈，最後讀13，共3個 block reads。不能同時把每塊當成4096 bytes user payload 又忽略嵌入指標。

FAT 把 next pointer 集中到獨立表，`FAT[11]=12,FAT[12]=13,FAT[13]=EOF`。FAT 全在 memory 時仍需2次 pointer lookup 才定位第3塊，但不用讀前兩個 data blocks，所以是1次 data I/O；FAT 冷時應另計它所跨的 table blocks。FAT 改善的是 chain metadata 的 locality，不會把追鏈的 CPU 步數從 $O(q)$ 自動改成 $O(1)$。indexed allocation 才能用第 q 個 index entry 直接定位。

extents 將一段表示成 `(logical_start,physical_start,length)`，在段內以差值運算；例如 `(0,8,6)` 直接把 logical4映到physical12。成長若遇到相鄰區域已占用，可新增 extent 而非一定搬移整個檔案，但 extent tree 的查找與 metadata 成本必須另計。

Free-space bitmap：位元約定與實際配置

OSC 9e 的 free-space bit vector約定 **1=free、0=allocated**。若 bitmap 覆蓋16 blocks，bytes/bit order 固定為 block0先：

```
block:  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
bits:   0 0 0 0 0 0 0 1 0 0  0  1  1  1  1  1
```

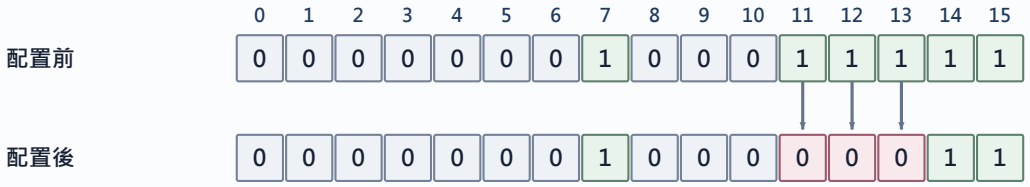
初始字串是 `0000000100011111`，free blocks={7,11,12,13,14,15}。inode7已有 data blocks8–10；append12 KiB需要三個 blocks。若要求單一連續 extent，first-fit從block11起選11、12、13，不選block7只因下一個block8已配置。

配置後：

```
bits: 0000000100000011
free: {7,14,15}
```

Free-space bitmap：位元約定與實際配置 （續）

first-fit配置三個連續blocks：11、12、13由free變used



inode7：blocks [8,9,10] → [8,9,10,11,12,13]；size 12KiB → 24KiB

綠1=free。灰0=原已配置。紅0=本次claim；剩餘free集合{7,14,15}。

順序必須維持 已配置 `block` 不在 `free set`；有效 `inode` 指向的 `block` 必為 `allocated`。先在volatile memory選位、再在disk bitmap與inode pointers先後更新，若中間 crash，可能造成double allocation或 lost/free orphan。記憶體 lock只解決並行 race，不提供斷電原子性。

bitmap overhead 可精算：

$$\#bits = \#blocks, \quad \#bytes = \left\lceil \frac{\#blocks}{8} \right\rceil.$$

1 TiB volume、4 KiB blocks有 2^{28} blocks，bitmap至少 2^{25} bytes=32 MiB；storage / filesystem I/O仍以 blocks 為單位。

同一 free set 的四種表示與更新

沿用 free set `{7,11,12,13,14,15}`。bitmap 保留16 bits；linked free list 可為 `head7→11→12→13→14→15→null`。若只配置一塊，可直接取 head7並令head=11，並非每次配置都要掃完整串列；找連續3塊才需要另外搜尋/整理。

Grouping 把數個 free addresses 放在一個管理 block，中間不必為每個 next 都讀一個磁碟 block。若每組存3個 addresses，前兩個為可配置頁、第三個連到下一組，則可用一筆讀取取得兩個候選；指向下一組的管理頁何時釋放，必須由組格式規定。

Counting 用 `(start,length)`，本例為 `(7,1),(11,5)`。配置連續3塊時，找到 `(11,5)`，回傳 `[11,14)`，剩下 `(14,2)`；若日後釋放 `[11,14)`，與相鄰的 `(14,2)` 合併回 `(11,5)`。不變量是各 free intervals 不重疊、相鄰者合併，union 恰為 free set。以平衡搜尋樹保存可做 $O(\log r)$ 的插入/合併，r 為 free runs 數；若還要按長度找足夠區間，需另外的 size index 或 subtree maximum，不能只靠 offset tree 保證所有查找都是對數。

ZFS space-map 的教材模型再把配置/釋放記為有序 extent log。初始`(11,5)`，重放`alloc(11,3)`後為`(14,2)`，再重放`free(12,1)`為`(12,1),(14,2)`。以 checkpoint+剩餘 log 重建，避免每次都在原位置改整片 bitmap；重放仍需正確的事件次序與交易識別，不能把同一 alloc 重放兩次當成兩次新配置。

Efficiency：cache、read-ahead 與 writeback

沿用 cold path：3 metadata reads+2 data reads=5 block reads。若 directory/inode cache全hit且data已在page cache，則本輪是0 disk reads；cache hit率必須按層報告，不能用「全部I/O減少80%」替代。

至少三個成本模型：

1. **cache hit**：省 disk transfer，但 lookup、copy、locking與 memory仍要成本。
2. **read-ahead**：把 block8、9接續載入，減少 seek/latency；猜錯則多讀。
3. **async writeback**：`write()`可先回傳，資料仍在 dirty page cache；完成與 crash durability另由 flush/commit記錄。

固定10,000個檔案**每個恰為33 KiB**、無壓縮/dedup/sparse，對比4 KiB與64 KiB allocation units。4 KiB時每檔需要9 units、配置36 KiB、tail waste3 KiB；64 KiB時需要1 unit、配置64 KiB、tail waste31 KiB。後者每檔多浪費28 KiB，合計280,000 KiB=273.4375 MiB。不能只給平均file size就對平均值取 ceiling，因一般 $E[\lceil S/B \rceil] \neq \lceil E[S]/B \rceil$。

在「每 unit 另外一個4-byte mapping entry」的簡化模型，entry 數9→1，只省32 bytes/file、全體320,000 bytes；這與28 KiB資料空間差額不是同一單位。真實 inode/extents/index blocks 的 metadata 要依結構另算，不能把少配置的資料 bytes 稱為省掉的 pointer bytes。

Pointer size、統一 page cache 與同步寫入

4-byte pointer 可表示 2^{32} 個 block位置；在4 KiB block下，單層 pointer範圍只是 $2^{32} \times 4KiB = 16TiB$ 的理論 addressability，實際 file size 受 direct/indirect結構、block count與filesystem欄位限制。8-byte pointer擴大地址範圍，但 index block能存放的 pointers減半，因此並不一定在同容量下更省空間。

unified page cache讓 mapped file與 read/write共用 page；一致性要靠 PTE、dirty tracking、writeback與 file locking協定。synchronous write 必須等指定 durability point；asynchronous write 可能在 cache 回傳，若之後斷電，成功 `write()`不足以證明內容持久。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Journal recovery：從故障點重建唯一結果

固定交易 T7 在原始檔案 size=12 KiB 的狀態追加 12 KiB：自選定起直到 commit 或 abort，blocks 11、12、13 由分配鎖或預留機制獨占，不供並行交易使用；舊資料塊不被覆寫。崩潰後先完成 journal recovery，才恢復分配或對外提供新狀態。

```
metadata: bitmap 11,12,13: free→used
          I7.blocks += 11,12,13
          I7.size = 24 KiB
data:     block11,12,13
```

本例使用 redo-only、no-steal 的 ordered metadata journal。descriptor、metadata after-images 與 commit 使用可辨識 transaction ID；假設完整 commit record 可原子辨識，torn/incomplete record 不算提交。持久寫入次序為 descriptor→data completion→metadata records durable→commit durable→checkpoint durable→clear transaction。commit 前不把未提交 metadata 寫回 home locations，且 flush/barrier 真正要求的持久順序不能被裝置 volatile cache 任意顛倒。故障前後：

Journal recovery：從故障點重建唯一結果 （續）

crash point	redo見到的狀態	recovery action	外部可見
descriptor前	無T7	discard	old，size12 KiB
descriptor後、commit前	未提交	丟棄	old
commit後、home未完成	已提交	冪等重放	new，size24 KiB
home完成、transaction未清	已提交且可能已套用	重放/辨識已完成	仍為new

「已提交」不能只看 data block寫了；commit record是 atomic transaction的判準。為避免metadata指到尚未寫完的data，本模型要求 data完整寫回後才寫commit（ordered metadata journal）。若只用metadata journal，user data的內容不自動具有同一個原子性或durability保證。

fsck可從directory/inode/bitmap掃描找出orphan block或重複配置，但無法推回一個從未持久寫入的使用者寫入；consistency與durability分開驗證。

Journal recovery：從故障點重建唯一結果 （續）



為何任一 crash prefix 都只恢復成舊或新

metadata 狀態用三元組 $M = (bitmap, pointers, size)$。舊狀態 M_0 的 pointers=[8,9,10]、size=12288；新狀態 M_1 的 pointers=[8,9,10,11,12,13]、size=24576。

commit 前。 no-steal 保證 home=M_0，新 data 雖可能已在11–13上，尚未由有效 inode引用；忽略未提交 log 後仍是 M_0，free bitmap可重新配置那些 blocks。

commit 後、checkpoint 未完成。 完整 after-images 已先持久，故任何部分 home 更新都可由 log 覆寫成 M_1。例如 home bitmap 已新而 inode pointers仍舊，不能直接對外開放此中間狀態；先 redo三個欄位，再 mount。因為 redo 是 `size←24576` 與 `pointers←完整新陣列`，重放兩次仍是 M_1，不是再 append一次變成36 KiB。

log 清除後。 清除只允許在 checkpoint durable 之後，故沒有 log 時 home 已完整 M_1。三分支涵蓋每個合法 crash prefix，證得原子 metadata 可見性。若允許 commit 前改 home，就需 undo 或其他協定，不能仍以「丟掉未提交log」證明恢復成功。

Journal、snapshot 與 backup 解不同故障

journal 解決中斷更新的結構一致性，不能恢復使用者合法刪除後已被覆寫的內容。snapshot 保留舊可達 blocks，可回到先前版本，但與原 volume 共用裝置，裝置完全損毀仍可能一起失去。backup 另外保存資料副本，恢復所需的來源必須實際存在。

固定星期日 full backup B0，週一 incremental B1只含週日後變動，週二 B2只含週一後變動。週三損毀要回到週二狀態，必須依 B0→B1→B2恢復，還要包含刪除/改名紀錄；只套 B0+B2會漏掉僅在週一改動的檔案。若 B2 是 differential，保存「自B0後所有變動」，才可 B0+B2。mtime 不可信或備份中來源仍在改寫時，需要 snapshot/一致性點與更可靠的變更記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFS：file handle、cache validation 與 retry

固定 **NFSv3（RFC1813）**，不把NFSv4的OPEN/session state混入。client先從mount取得遠端root handle，再以`LOOKUP(directory_handle,name)`逐段取得opaque file handle；READ明列handle、offset與count。NFSv3沒有遠端OPEN/CLOSE程序，client的fd/offset管理不等於server保存一個POSIX open description。handle識別的是物件，不是可以任意解析的檔名。

例如已取得 `/export` 的H0，`LOOKUP(H0,"u")→H1`，`LOOKUP(H1,"notes.txt")→H7`，再`READ(H7,offset=4096,count=4096)`。相同READ可重送而不重複修改server狀態，但若期間其他client寫檔，重讀內容未必等於第一次回覆；「副作用冪等」不等於「結果永不改變」。REMOVE回覆遺失後，另一client若在同名建立新物件，再把舊REMOVE當新請求執行便可能刪錯物件。

固定 retry trace：

step	client	server	保存證據
1	send create X id=12	建立 X，回覆遺失	server request id/effect/result
2	timeout，重送 id=12	找到舊紀錄	回傳原結果，不再建立

這需要 duplicate-request cache或對應版本的 session/transaction機制，且保存 duration 與 crash behavior要明示。有限 cache過期後，不能虛稱永久 exactly-once。client cache要驗證版本/mtime/file handle；cache hit不是server commit。

UNSTABLE write、COMMIT 與 server 重啟

NFSv3的WRITE回覆含實際承諾的穩定程度與write verifier。client要求UNSTABLE時，server可先只保留volatile資料；client在需要持久性時發COMMIT，並檢查verifier與先前WRITE一致。若server重啟遺失volatile資料，verifier改變，client必須重送仍未確認穩定的資料。

事件	server狀態	client尚需保留什麼
WRITE offset0,data=A，回UNSTABLE,v7	A可能只在RAM	A與未提交範圍
server重啟，回覆verifier改v8	A可能遺失	不能以先前WRITE成功視為持久
COMMIT回v8，與v7不同	不能證明A已保存	重送A，再要求穩定完成
重送WRITE後成功穩定完成，verifier一致	A已達協定穩定儲存承諾	可解除這段重送義務

client attribute/data cache還可能暫時陳舊。close-to-open或timeout validation是特定client的快取政策，不能從NFSv3名稱推成全域線性一致。遠端mount也不自動繼承server其餘mounts；namespace要依client自己的mount table解析。

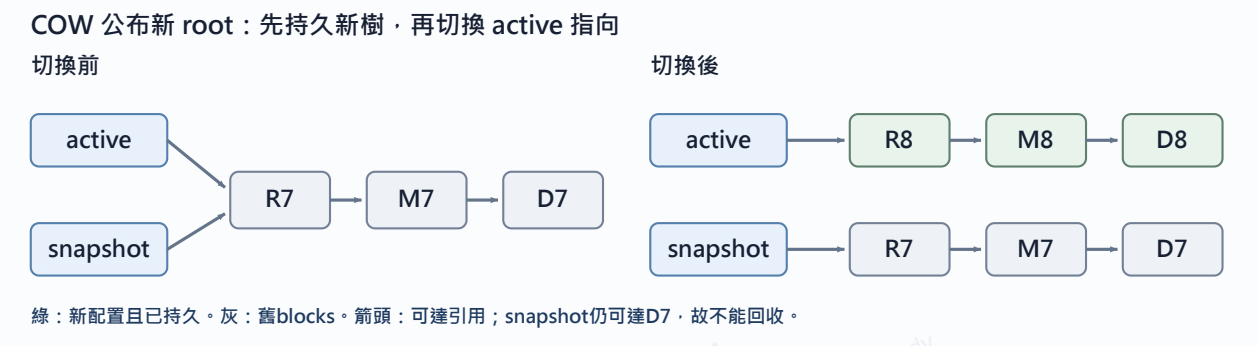
WAFL COW：root 切換與 snapshot 共享

固定 version7 使用 root R7→metadata M7→data D7，active root與snapshot root都可到達舊樹。下表的「2個保留root可達」是教學用的可達性計數，**不是斷言WAFL對每個block實作整數refcount=2**；教材描述的snapshot bitmap可記錄哪些快照仍使用該block。修改 D7 的流程：

stage	active root	新 blocks	舊樹的可達性
v7	R7	—	active與snapshot都可達R7/M7/D7
prepare v8	仍是R7	D8,M8,R8	R8由交易保留，尚未公布；舊樹仍可達
durable	仍是R7	D8/M8/R8已持久	crash before switch仍讀v7
atomic switch	R7→R8	切換後	active可達新樹，snapshot仍可達舊樹

COW寫入把新資料放到新 block，所有新data/metadata先持久，最後以可復原的atomic root publication切換可見版本；不要求真實裝置任意大小的root更新都天然原子。snapshot保留R7時，D7仍可達而不能回收；刪掉snapshot且沒有其他root引用後才具備回收條件。準備中未公布的blocks也須由交易的allocation管理保留，不可因尚無public root就被別人重用。

WAFL COW：root 切換與 snapshot 共享 （續）



這增加 write amplification：一個 data block更新可能同時需要新 data、metadata與root blocks。WAFL的 free-space collector需同時追蹤 active/snapshot refs，不能只因 block標成「舊」就回收。

從名稱到block：需要維持的四組對應

對應	一般不變量	失敗例
path／directory→inode	依namespace解析至實際物件	symlink suffix被字串先消掉
fd→OFD→inode	offset／reference歸屬正確	dup後誤以為offset獨立
logicalblock→physicalblock／hole	實際配置與資料位置一致	hole被當作disk block0
pointer／size↔free-space metadata	已配置block不被第二owner重配	bitmap先free但舊pointer仍可用

memory lock維持同時操作的互斥；crash後的多項metadata一致性另需journal／COW等更新協定。cache valid、dirty、writeback及device completion也不能用一個「已寫」合併。

journal與COW的提交時點回到同一個問題

T7 使用 redo-only／no-steal；commit durable 後可重放完整 after-images。

crash prefix	no-steal redo模型的恢復
commit尚未durable	home仍old，不能把未提交更新先寫home
commit durable、home部分更新	重放全部after-images得到一致new
home全部durable、checkpoint完成	按已證安全截斷條件回收log

after-image賦值可冪等重放；若允許未提交home先落盤，便須額外undo等機制，不能沿用本表。COW則先持久化新data／metadata，再以可恢復的atomic root publication切換；old snapshot仍可達的blocks不能先回收。

metadata 一致不等於完整資料交易、備份或 NFS 跨 client 一致性。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study